

Wydział Informatyki

Informatyka

Data 11-05-2026

Rok II Semestr IV Zespół: Mateusz Tręda Dominik Jarzyło	Prowadzący: prof. dr hab. Piotr Hońko
--	--

DEKLARACJA PROJEKTU

Aplikacja do analizy meczy Dota 2

Baza danych Oracle • SQL Developer

1. Zakres Merytoryczny

Aplikacja stanowi system analityczny dedykowany grze Dota 2 — wieloosobowej strategii czasu rzeczywistego (MOBA) firmy Valve. Jej głównym celem jest gromadzenie, przetwarzanie i prezentowanie statystyk rozgrywek w sposób ułatwiający analizę stylu gry, postępów oraz trendów meta.

Główne obszary analityczne

- Analizy meczów — szczegółowe dane o rozgrywkach (czas trwania, składy drużyn, wyniki KDA, netto-worth)
- Analizy przedmiotów — popularność oraz skuteczność przedmiotów w zależności od bohatera i rangi
- Analizy bohaterów — statystyki wyboru (pick rate), skuteczności (win rate) i wydajności per pozycja
- Profile graczy — historia rozgrywek, ewolucja rangi, ulubieni bohaterowie
- Śledzenie zmian meta — wpływ patchów (buff/nerf) na popularność i skuteczność bohaterów

2. Elementy Słownikowe

System wykorzystuje następujące słowniki jako listy wartości dozwolonych (CHECK constraints w bazie danych):

Słownik	Dozwolone wartości	Opis
RANGA	Herald, Guardian, Crusader, Archon, Legend, Ancient, Divine, Immortal	8-stopniowa drabinka rankingowa MMR w Dota 2
POS	1, 2, 3, 4, 5	Pozycja gracza na mapie: 1=Carry, 2=Mid, 3=Offlane, 4=Soft Supp, 5=Hard Supp
ATR	Strength, Agility, Intelligence, Universal	Główny atrybut bohatera wpływający na skalowanie statystyk
TYP MECZU	Ranked, Normal	Określa, czy mecz wlicza się do rankingu MMR gracza
PATCH STATUS	Buff, Nerf, Neutral	Źródło: oficjalne noty do patcha Valve; aktualizowane po każdej wersji gry

Lista Encji

Najważniejsze modele w bazie danych:

- Bohater - Postać kontrolowana przez gracza w rozgrywce. Posiada 3 główne atrybuty: Siła, Zręczność, Inteligencja,
- Przedmiot – pozwala bohaterowi podnieść statystyki i zapewnia dodatkowy czar.
- Mecz – rozgrywka 5 na 5 graczy bo obu stronach konfliktu: Świetliści kontra Mroczni,
- KDA - trójka wartości liczbowych. Kill Death Assist - drogą do wygrania meczu jest zabijanie wrogich bohaterów w celu dalszego rozwoju postaci i przewagi drużynowej,
- Zagranie Bohatera – opisuje, na jakiej pozycji gracz grał wybranym bohaterem i jakie wykorzystał przedmioty.

3. Historia i Przechowywanie Danych

Strategia archiwizacji

System przechowuje pełną historię rozgrywek z możliwością agregacji do różnych przedziałów czasowych. Dane historyczne są składowane warstwowo:

- ☐ Hot data (ostatnie 30 dni) — pełna szczegółowość, bezpośredni dostęp z tabel operacyjnych
- ☐ Warm data (3–12 miesięcy) — agregaty tygodniowe, kompresja starszych rekordów meczów

- ❑ Cold data (archiwum > 1 rok) — agregaty miesięczne, przechowywanie w tabelach MATCH_ARCHIVE

Agregaty i trendy

- ❑ Dzielne: liczba meczów, średni czas trwania, top-10 bohaterów
- ❑ Tygodniowe: win rate na bohatera/rangę, popularność przedmiotów, średnie KDA graczy
- ❑ Miesięczne: ewolucja meta (buff/nerf impact), wzrost bazy graczy per ranga, trendy pick/ban
- ❑ Sezonowe: porównanie patchów, ranking najskuteczniejszych bohaterów w danym sezonie

4. Rozmiar Bazy Danych i Przewidywany Przyrost

Szacunkowy przyrost danych na podstawie aktywności typowej platformy analitycznej:

Encja	Początkowy rozmiar	Przyrost / miesiąc	Tempo wzrostu	Uwagi
Match_Game + Hero_Played	0	~150000 rekordów	Szybki	10 wierszy HP na mecz
Player	40 (seed)	~2000 nowych graczy	Normalny	Rośnie z bazą użytkowników
Item	~130 (seed)	~5–10 / patch	Wolny	Aktualizacja co 2–3 miesiące
Hero	~124 (seed)	<5 / rok	Rzadki	Nowy bohater co kilka miesięcy

Przewidywany rozmiar po 12 miesiącach eksploatacji: ~2 GB danych operacyjnych + ~500 MB agregatów historycznych.

5. Bezpieczeństwo i Ochrona Danych

Haszowanie haseł i danych wrażliwych

- ❑ Hasła użytkowników przechowywane wyłącznie w postaci skrótu bcrypt (salt rounds)

Ochrona przed SQL Injection

- ❑ Wszystkie zapytania realizowane przez Oracle Prepared Statements / Bind Variables (:param)
- ❑ Walidacja po stronie Backend'u. Funkcja w PHP htmlspecialchars

- Uprawnienia bazy: konto aplikacyjne (APP_USER) posiada wyłącznie SELECT/INSERT/UPDATE na wybranych tabelach — brak DDL

Użytkownicy

- dota_app - Użytkownik posiadający najwięcej uprawnień. Odpowiada za komunikację z backendem. Ma uprawnienie do utworzenia meczu za pomocą kreatora administracyjnego.
- Gracz – widzi swoje statystyki i analizę swoich meczy. Może zobaczyć innych użytkowników
- Gość - widzi jedynie statystyki globalne, nie wie jacy gracze i jakie mecze za to odpowiadają.

6. Modelowanie Danych

Relacyjny model logiczny

System oparty jest na znormalizowanym modelu relacyjnym (3NF). Poniżej zestawienie tabel wraz z kluczami i więzami:

Tabela	Klucz główny (PK)	Klucze obce (FK)	Więzy integralności
Item	id (seq)	—	UNIQUE(name), NOT NULL
Hero	id (seq)	—	UNIQUE(name), CHECK(primary_attribute)
Player	steam_id	—	CHECK(rank), NOT NULL(nickname, date)
Hero_Played	id (seq)	steam_id→Player, hero_id→Hero, slot1-6→Item	CHECK(position 1-5), VIRTUAL KDA
Team	id (seq)	hp1- 5→Hero_Played	CHECK(side), UNIQUE(hp1-5)
Match_Game	id (seq)	team1_id, team2_id, winner_id → Team	CHECK(winner ∈ {team1,team2})

Diagram fizyczny — indeksy

Następujące indeksy są zalecane na etapie implementacji fizycznej:

- ☐ Hero_Played: INDEX(steam_id), INDEX(hero_id), INDEX(position) — częste filtrowanie
- ☐ Match_Game: INDEX(match_time DESC), INDEX(is_ranked), INDEX(winner_id) — sortowanie i filtr
- ☐ Player: INDEX(rank) — zapytania top-N per ranga
- ☐ Hero: INDEX(primary_attribute) — filtrowanie po atrybucie
- ☐ Klucze główne otrzymują indeks automatycznie (Oracle B-Tree na PK)

Tabelami o największym obciążeniu są Mecze ponieważ każdy gracz zechce wyciągnąć pojedynczy mecz do własnej analizy. Znacząco będą też napływać nowe rekordy do tabeli ponieważ w tą grę gra wiele osób.

7. Cache — Częste Kwerendy

Zapytania o wysokiej częstotliwości powinny być cachowane z określonym TTL:

Kwerenda	TTL	Uzasadnienie
Top 20 bohaterów wg win rate (dana ranga)	1 godz.	Zmienia się rzadko, wysoki ruch
Statystyki gracza (ostatnie 30 meczów)	10 min.	Średnio aktywny gracz nie gra częściej
Pick rate / ban rate (tydzień)	30 min.	Agregat tygodniowy — stabilny
Lista all Heroes + atrybut	24 godz.	Słownik — zmienia się tylko przy patchu
Zbuffowani bohaterowie (ostatni patch)	24 godz.	Statyczne do następnego patcha

8. Wymagania Funkcjonalne i Niefunkcjonalne

8.1 Wymagania funkcjonalne

WF-01 Zestawienie statystyk jednego gracza

- ☐ System wyświetla dla podanego steam_id: liczbę meczów, średnie KDA, win rate, najczęściej granych bohaterów, netto-worth na mecz
- ☐ Dane filtrowane po przedziale czasowym i typie meczu (Ranked / Normal)

WF-02 Wyszukiwanie gracza

- ☐ Wyszukiwanie po nicku (częściowe dopasowanie LIKE) lub dokładnym steam_id
- ☐ Wynik zawiera: nickname, rangę, datę założenia konta, podsumowanie ostatnich 10 meczów

WF-03 Top bohatera na rangę

- ☐ Ranking bohaterów według win rate / pick rate dla wybranej rangi (słownik RANGA)
- ☐ Filtrowanie po pozycji (POS 1–5) i atrybucie głównym (ATR)

WF-04 Zbuffowany bohater (patch tracker)

- ☐ Lista bohaterów z flaga Buff/Nerf/Neutral dla bieżącego patcha
- ☐ Porównanie win rate przed i po patchu (delta procentowa)

WF-05 Filtrowanie zaawansowane

- ☐ Filtrowanie po: win rate %, pick rate %, ranga, pozycja, atrybut, typ meczu, patch
- ☐ Sortowanie i stronicowanie wyników (paginacja)

8.2 Wymagania niefunkcjonalne

Kategoria	Wartość docelowa	Uzasadnienie
Czas odpowiedzi (cache hit)	< 200 ms	Kwerendy słownikowe i top-N
Czas odpowiedzi (cache miss)	< 2 s	Agregaty historyczne bez cache
Dostępność systemu	99,5% / miesiąc	Poza oknami konserwacyjnymi
Obsługa jednoczesnych użytkowników	min. 100 sesji	Początkowa skala projektu
Bezpieczeństwo	0 wycieków PII	RODO, bcrypt, bind variables
Skalowalność bazy	obsługa 10M rekordów Hero_Played	Partycjonowanie po match_time

9. Potencjalne trudności

Trudnością może okazać się przesył rekordów w tabeli Hero_Played. Będą one występować tyle samo razy, ile gracz rozegra mecz wybranym bohaterem. Problematiczne może być również inteligentne zaprojektowanie versionowania danych dotyczących przedmiotów oraz bohaterów w celu wiernego odtwarzania statystyk po przeprowadzeniu zmian (buffów i nerfów) w grze, a także archiwizacji dawnych meczy.

10. Przewidywane Triggery, Widoki, Pakiety, Wyjątki

Pakiet GeneralInfo zawierałby funkcje zwracające podstawowe, aktualne informacje oraz obliczający ogólne statystyki, np.:

- META() - zwraca aktualną Metę czyli coś się ogólnie gra żeby zapewnić sobie duży winrate
- LatestPatch() - zwraca ostatnie zmiany w bohaterach i przedmiotach
- GetNumMatches(Date)
- AvgMatchDuration(Date)
- AvgKDA(StartDate, EndDate)
- GetItemPopularity(Item, StartDate, EndDate)
- RankItemsByPopularity(Item, StartDate, EndDate)
- PlayerGrowth(Rank, StartDate, EndDate)

Pakiet PlayerStatistics odpowiadałby za wyznaczanie statystyk i informacji istotnych dla danego gracza:

- QueryLastMatches(PlayerId, Count, Offset, MatchType, StartDate, EndDate)
- PlayerWinrate(PlayerId, Hero, Rank, MatchType, StartDate, EndDate)
- AvgKDA(PlayerId, MatchType, StartDate, EndDate)
- MostFrequentlyPlayedHeros(PlayerId, count, MatchType, StartDate, EndDate),
- NettoWorth(PlayerId, MatchId)

W pakiecie PlayerQuery będą znajdować się funkcje służące do wyszukiwania i pobierania ogólnych danych dotyczących wybranych graczy:

- SearchByNick(Nick, Count, Offset)
- GetPlayerInfo(PlayerId)

Pakiet HeroStatistics odpowiadałby za wyznaczanie statystyk dotyczących bohaterów:

- RankByWinRate(HeroRank, Pos, Atr, StartDate, EndDate)
- RankByPickRate(HeroRank, Pos, Atr, StartDate, EndDate)
- CalculateHeroWinRate(Hero, HeroRank, StartDate, EndDate)
- CalculateHeroPickRate(Hero, HeroRank, StartDate, EndDate)
- CalculateHeroBanRate(Hero, HeroRank, StartDate, EndDate)
- GetCharactersWithFlag(PatchId, Flag)
- CalculateWinRateDelta(Hero, PatchId)

Pakiet DatabaseAdministration zawierałby procedury wstawiające i aktualizujące dane w tabelach:

- RegisterMatch(...)
- AddNewHero(...)
- AddNewItem(...)
- NewPatch() – rejestruje wydanie nowego Patcha,
- PatchHero(...) – procedura mogłaby jednocześnie automatycznie określać typ zmiany: nerf lub buff
- PatchItem(...)

- ArchiveCold() – automatycznie archiwizuje najstarsze (ponad roczne) dane
- ArchiveWarm() – automatycznie archiwizuje nieco młodsze dane (ponad miesięczne)

Przewidywane są również wyzwalacze:

- OnHeroUpdateTrigger – odpowiada za automatyczne wersjonowanie danych dotyczących bohatera. Archiwizuje dotychczasowe dane i nadaje nowy numer wersji dla aktualnych danych
- OnItemUpdateTrigger – analogicznie do powyższego odpowiada za automatyczne wersjonowanie danych dotyczących przedmiotów.

11. Szczegółowe Wymagania Klienta

Poniżej zebrane wymagania zgłoszone bezpośrednio przez klienta w formie przypadków użycia:

WK-01 Filtrowanie po wskaźnikach

- ☐ Użytkownik może filtrować bohaterów według: win rate (%), pick rate (%), ban rate (%), średnie KDA, netto-worth
- ☐ Filtry łączne (AND) z możliwością ustawiania zakresów (np. win rate > 55%)
- ☐ Wyniki sortowalne po dowolnej kolumnie rosnąco / malejąco

WK-02 Wyszukiwanie gracza

- ☐ Pole wyszukiwania obsługuje: pełny nick, częściowy nick (LIKE '%...%'), steam ID
- ☐ Autouzupełnianie sugestii po wpisaniu min. 3 znaków (top 5 meczem podobnych nicków)
- ☐ Profil gracza zawiera: rangę, historię rang (timeline), ulubioną pozycję, ulubionego bohatera

WK-03 Top bohater na rangę

- ☐ Widok rankingowy: TOP 10 bohaterów dla wybranej rangi i pozycji
- ☐ Wyświetlanie zmiany pozycji w rankingu względem poprzedniego tygodnia (+/-)
- ☐ Eksport wyników do CSV

WK-04 Zbuffowany bohater

- ☐ Automatyczna etykieta BUFF / NERF / NEUTRAL po wczytaniu not do patcha
- ☐ Dashboard: bohaterowie z największym wzrostem win rate po ostatnim patchu
- ☐ Wykres trendu win rate bohatera na przestrzeni ostatnich 5 patchy